

SymPy Bug Report

1 Bug 1: solveset returns 0 for asin outside the principal range

1.1 Minimal Reproducer

```
import sympy
from sympy import Eq, S, asin, pi, simplify, solveset, symbols

x = symbols("x")
eq = Eq(asin(x), pi)
sol = solveset(eq, x, domain=S.Complexes)

print("SymPy version:", sympy.__version__)
print("solveset result:", sol)
print("residual at returned value 0:",
      simplify(eq.lhs.subs(x, 0) - eq.rhs.subs(x, 0)))
```

1.2 Observed Behavior

```
SymPy version: 1.14.0
solveset result: {0}
residual at returned value 0: -pi
```

SymPy returns the finite solution set $\{0\}$. However, direct substitution of the returned value into the original equation gives

$$\sin(0) - \pi = -\pi \neq 0.$$

Thus the returned element does not satisfy the equation that `solveset` was asked to solve.

1.3 Expected Behavior

The mathematically correct result is $\emptyset$, represented in SymPy as `EmptySet`. There is no complex value x for which SymPy's principal inverse sine satisfies $\sin(x) = \pi$.

1.4 Mathematical Explanation

SymPy's `asin` is the principal inverse sine. On real inputs its image is $[-\pi/2, \pi/2]$, and more generally the complex principal branch has a fixed principal image rather than all complex values. The target π is outside that principal image.

Applying `sin` to both sides gives $x = \sin(\pi) = 0$, but this implication is not reversible for the principal inverse sine. The value $x = 0$ satisfies $\sin(\pi) = 0$, but it does not satisfy the original equation because $\sin(0) = 0$, not π . The wrong answer is therefore not a harmless branch choice, a missing constant, or an equivalent representation of a solution; it is a concrete non-solution.

1.5 Source-Level Diagnosis

The diagnosis identifies the relevant code path in `sympy/solvers/solveset.py`, in the complex inversion helper `_invert_complex` near line 550. The traced call path is that `solveset` converts `Eq(asin(x), pi)` to `asin(x) - pi = 0` and enters `_solveset`. The expression is not handled by `_solve_trig`, because the top-level function is the inverse trigonometric function `asin`, not `sin`. The solver then calls `_invert(..., domain=S.Complexes)`, which dispatches to `_invert_complex`.

For this equation, `_invert_complex(asin(x) - pi, {0}, x)` removes the additive constant and then handles `asin(x)` through the generic function-inverse path. It applies `asin.inverse()`, i.e. `sin`, to the right-hand side and produces $(x, \{\sin(\pi)\}) = (x, \{0\})$. The problematic source rule is the generic use of a function's inverse when one is available:

```
return _invert_complex(f.args[0],
                       imageset(Lambda(n, f.inverse()(n)), g_ys), symbol)
```

For principal `asin`, this treats `sin` as a globally valid inverse without carrying the required range condition on the right-hand side. The later filtering accepts the finite inversion result after `domain_check` rather than checking the residual $\text{asin}(0) - \pi$ against the original equation. Consequently, the candidate 0 is accepted even though it is not a solution. A plausible fix direction supported by the diagnosis is to make inverse-function handling for principal inverse functions carry image or range conditions, returning $\emptyset$ or a `ConditionSet` when a concrete target such as π violates the principal `asin` image.

1.6 Additional Failing Instances

The same failure occurs for integer multiples of π that lie outside the principal image of `asin`. The independent verification checks

$$\text{asin}(x) = n\pi, \quad n \in \{1, 2, -1, 3, 4, 5\},$$

over the complex numbers. In each case, SymPy's inversion computes $x = \sin(n\pi) = 0$, but the returned value has residual $\text{asin}(0) - n\pi = -n\pi \neq 0$. Since none of these targets belongs to the principal image of `asin`, the expected solution set is $\emptyset$ in all cases.

Input / Expression	SymPy behavior	Expected behavior
<code>solveset(asin(x) = π, x, $\mathbb{C}$)</code>	returns $\{0\}$; residual $-\pi$	$\emptyset$
<code>solveset(asin(x) = 2π, x, $\mathbb{C}$)</code>	returns $\{0\}$; residual -2π	$\emptyset$
<code>solveset(asin(x) = $-\pi$, x, $\mathbb{C}$)</code>	returns $\{0\}$; residual π	$\emptyset$
<code>solveset(asin(x) = 3π, x, $\mathbb{C}$)</code>	returns $\{0\}$; residual -3π	$\emptyset$
<code>solveset(asin(x) = 4π, x, $\mathbb{C}$)</code>	returns $\{0\}$; residual -4π	$\emptyset$
<code>solveset(asin(x) = 5π, x, $\mathbb{C}$)</code>	returns $\{0\}$; residual -5π	$\emptyset$

1.7 Related Issues Assessment

The bug-finding harness performed a best-effort deduplication check. The closest related public material found was documentation confirming the principal-branch behavior of `asin`, general

principal-branch references, and SymPy documentation stating that `solveset` returns values for which the equation is true. No public issue, pull request, discussion, or external report was identified for $\operatorname{asin}(x) = \pi$, the wrong output $\{0\}$, or a missing principal-range check in `solveset` for this exact case. The deduplication verdict is therefore a new instance of a known failure mode: the general need for branch- and range-aware inverse solving is known, but this specific reproducible case appears previously unreported and remains individually actionable as an issue and regression test.

1.8 Proposed SymPy Regression Test

```
import pytest

from sympy import Eq, FiniteSet, S, asin, pi, simplify, solveset, symbols

@pytest.mark.parametrize("n", [1, 2, -1, 3, 4, 5])
def test_solveset_principal_asin_rejects_out_of_range_targets(n):
    x = symbols("x")
    eq = Eq(asin(x), n*pi)
    sol = solveset(eq, x, domain=S.Complexes)

    assert sol != FiniteSet(0)
    assert simplify(eq.lhs.subs(x, 0) - eq.rhs.subs(x, 0)) != 0
```